EventPlanner — System Design Document

1. Executive Summary

The EventPlanner system is a scalable, cloud-native web application designed to facilitate the creation, discovery, and booking of events. Built on a Microservices Architecture using Spring Boot, the system decouples core functionalities into independent services to ensure maintainability, fault isolation, and independent scalability.

The high-level architecture consists of five core microservices: User Service (Identity), Event Catalog Service (Inventory), Booking Service (Orchestration), Payment Service (Transactions), and Notification Service (Async Messaging). The system leverages Spring Cloud Netflix Eureka for service discovery, Spring Cloud Gateway for routing, and RabbitMQ for asynchronous communication between booking and notification processes.

- **Project name:** EventPlanner
- **Authors / Team:**

Rauf Kutay Akyıldız - Booking Service
Samet Laçin - User Service
Erdoğan Efe Güner - Payment Service
Selim Can Aydın - Event Catalog Service
Ömer Yiğit Kartal - Notification Service

- **Target users / stakeholders:** Event Organizers (create events), Attendees (book tickets), System Administrators (manage platform)
- **Scope:** This document covers the backend architectural design, API specifications, data models, and deployment strategy for the microservices on Render. Frontend implementation details are excluded from this scope.

---

2. Goals & Non-Functional Requirements (NFRs)

List measurable goals and NFRs such as:

- **Availability:** The system aims for 99.9% uptime during business hours, achieved through service discovery (Eureka) and fault isolation provided by the microservices architecture.
- **Scalability:** The system must support horizontal scaling, specifically allowing the Event Catalog Service to handle up to 1,000 concurrent read requests independently from other services.
- **Performance:**

Average API response time for read operations (e.g., listing events) should be < 300 ms.

Asynchronous notifications (via RabbitMQ) should be processed within 5 seconds of the trigger event.

- **Security:**

*** Authentication:** All protected endpoints must be secured using JWT (JSON Web Tokens) issued by the User Service.

***Transport Security:** All data in transit must be encrypted using TLS/HTTPS.

***Secrets Management:** Database credentials and API keys must be managed via environment variables on the Render platform.

- **Maintainability:** The system must adhere to the Database per Service pattern to ensure loose coupling and independent deployability of microservices.
- **Data Retention & Compliance:** Application logs and transaction history should be retained for 90 days for auditing and debugging purposes.

3. Scope and Requirements

3.1 Functional Requirements

Use numbered items and map them to microservices where appropriate.

1. Users can create an account, login, and obtain a JWT for authentication (**User Service**).
2. Organizers can create and update event listings, including capacity and location details (**Event Catalog Service**).
3. Users can browse available events and view current stock status (**Event Catalog Service**).
4. Users can book tickets; the system orchestrates validation across user, event, and payment services (**Booking Service**).
5. Payments are processed (simulated) and transaction logs are recorded (**Payment Service**).
6. Users receive an asynchronous email confirmation automatically after a successful booking (**Notification Service**)

3.2 Constraints & Assumptions

1.The system must be implemented using the Spring Boot framework.

2.Each microservice should be maintained in its own Git repository (or a structured mono-repo) to ensure loose coupling.

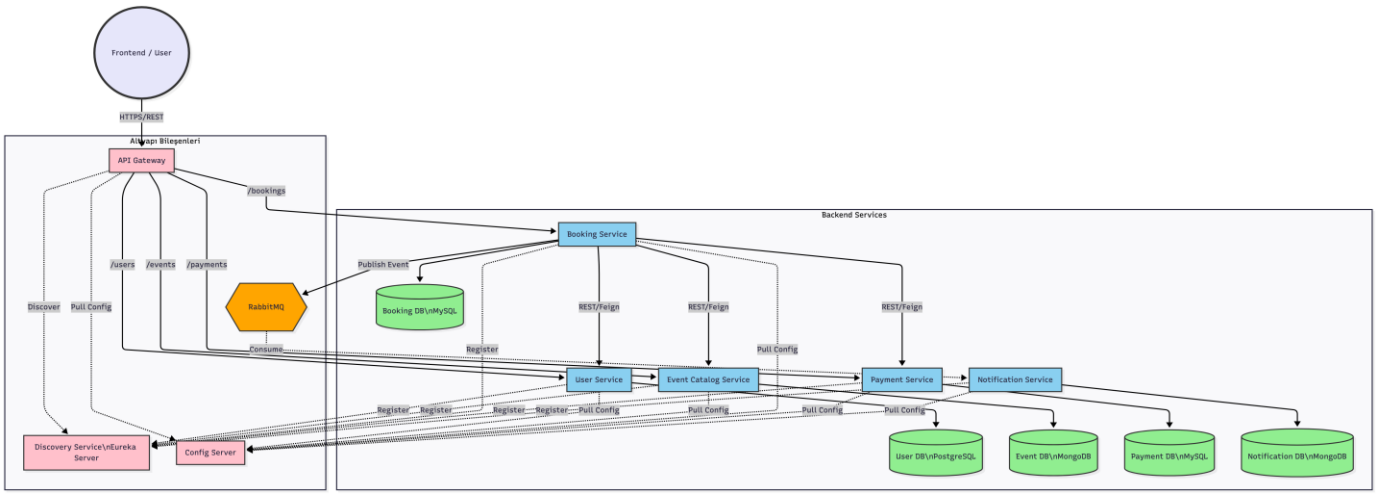
3.The application must be deployed to the Render cloud platform as independent web services.

4.Docker is assumed to be installed for local testing of infrastructure components (e.g., RabbitMQ, databases).

5.The Payment Service simulates financial transactions; no real banking integration is required.

---

4. High-Level Architecture



Architecture Overview: The system follows a distributed microservices pattern designed for scalability and fault tolerance.

- **Entry Point:** The API Gateway acts as the single entry point, routing requests (e.g., /bookings, /users) to the appropriate backend services.

- **Service Discovery:** Netflix Eureka is used for dynamic service registration, allowing services to find each other without hardcoded IPs.
- **Configuration:** Spring Cloud Config Server manages centralized configuration for all microservices.
- **Orchestration:** The Booking Service acts as the central orchestrator, communicating synchronously via REST/Feign with the User, Event, and Payment services to ensure data consistency.
- **Asynchronous Processing:** RabbitMQ handles non-blocking communication between the Booking Service and the Notification Service, ensuring user experience is not impacted by email delivery latencies.

---

5. Component Design (Per Service)

For each microservice, create a subsection with the following structure. Duplicate the subsection per service.

5.1 **User Service**

- **Purpose:** Handles user identity management, secure registration, and authentication processes.
- **Responsibilities:**
  - Register new users and securely hash passwords (BCrypt).
  - Authenticate users and issue JWT (JSON Web Tokens).
  - Validate tokens for other services (Inter-service security).
  - Manage user profile information.
- **Exposed APIs:**
  - `POST /auth/register` (Create account)
  - `POST /auth/login` (Get JWT)
  - `GET /users/{id}` (Get profile)
- **Data storage: PostgreSQL** (`users` table) — Chosen for strict relational integrity and ACID compliance required for identity data.
- **Dependencies:** Config Server, Eureka Client.
- **Scaling strategy:** Stateless architecture; can be scaled horizontally behind the Gateway.

5.2 **Event Catalog Service**

- **Purpose:** Manages the creation, listing, and inventory tracking of events.
- **Responsibilities:**
  - Allow organizers to create and update events.
  - Serve high-performance read queries for event listings.
  - Manage ticket stock (concurrency handling for `availableSeats`).

- **Exposed APIs:**
    - `GET /events` (List all events)
    - `GET /events/{id}` (Get details)
    - `POST /events` (Create event - Organizer only)
    - `PUT /events/{id}/stock` (Internal: Update stock)
- **Data storage: MongoDB** (`events` collection) — Chosen for flexible schema (different event types) and high read throughput.
- **Dependencies:** Config Server, Eureka Client.
- **Scaling strategy:** Horizontal scaling with database sharding (if needed) for high read loads

5.3 **Booking Service**

- **Purpose:** Acts as the central orchestrator for the ticket purchasing workflow.
- **Responsibilities:**
    - Coordinate the booking process (Saga pattern).
    - Communicate synchronously with User (validation), Event (stock), and Payment services.
    - Manage booking states (`PENDING`, `CONFIRMED`, `CANCELLED`).
    - Publish `BookingCreatedEvent` to RabbitMQ for asynchronous notifications.
- **Exposed APIs:**
    - `POST /bookings` (Place order)
    - `GET /bookings/{id}` (Check status)
- **Data storage: MySQL** (`bookings` table) — Relational DB used to ensure transactional consistency of orders.
- **Dependencies:** User Service, Event Catalog Service, Payment Service (via Feign Client), RabbitMQ, Config Server.
- **Scaling strategy:** Stateless service; relies on external database and message broker for state.

5.4 **Payment Service**

- **Purpose:** Isolates financial transaction logic and simulates banking integrations.
- **Responsibilities:**
    - Process payment requests securely.
    - Record transaction logs and status (`SUCCESS`/`FAILED`).

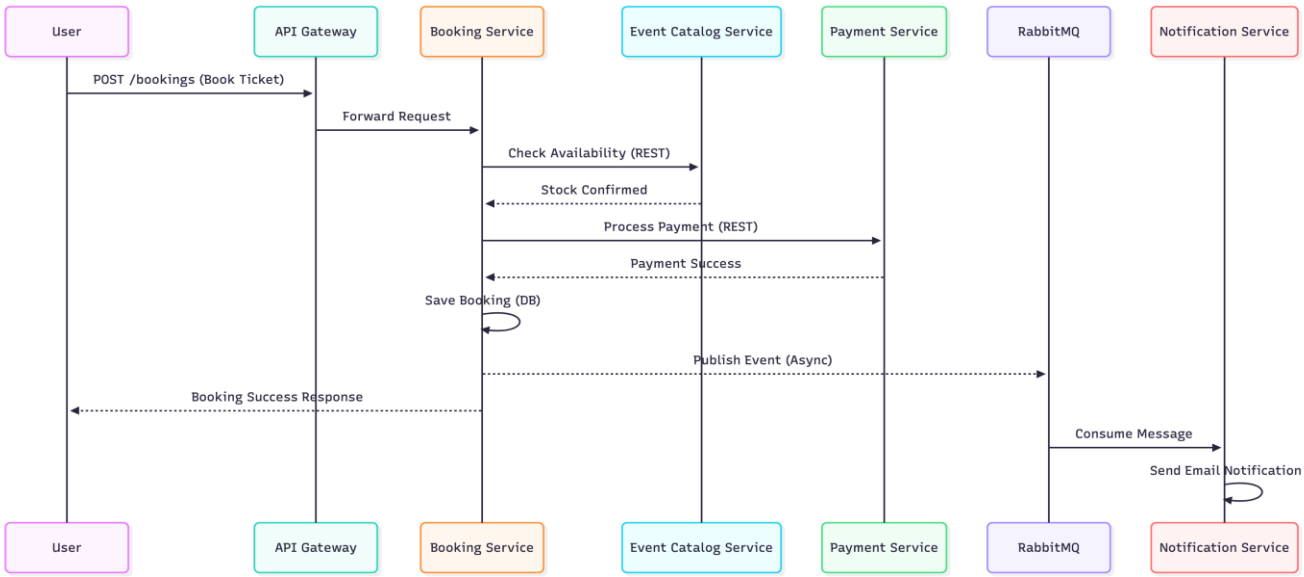
    - Simulate external payment gateway latency/responses.
- **Exposed APIs:**
    - `POST /payments` (Process transaction)
- **Data storage: MySQL** (`transactions` table).
- **Dependencies:** Config Server, Eureka Client.
- **Scaling strategy:** Horizontal scaling.

5.5 **Notification Service**

- **Purpose:** Handles asynchronous user alerts to decouple communication latency from the main booking flow.
- **Responsibilities:**
    - Listen to and consume messages from `notificationQueue` (RabbitMQ).
    - Generate and send email confirmations (Simulated JavaMailSender).
    - Log notification history for audit purposes.
- **Exposed APIs:** None (Message Consumer).
- **Data storage: MongoDB** (`notification_logs` collection) — ideal for unstructured log data.
- **Dependencies:** RabbitMQ, Config Server.

- **Scaling strategy:** Can scale the number of consumers based on queue depth (backpressure handling).

6. Example Sequence Diagram — Ticket Booking Flow



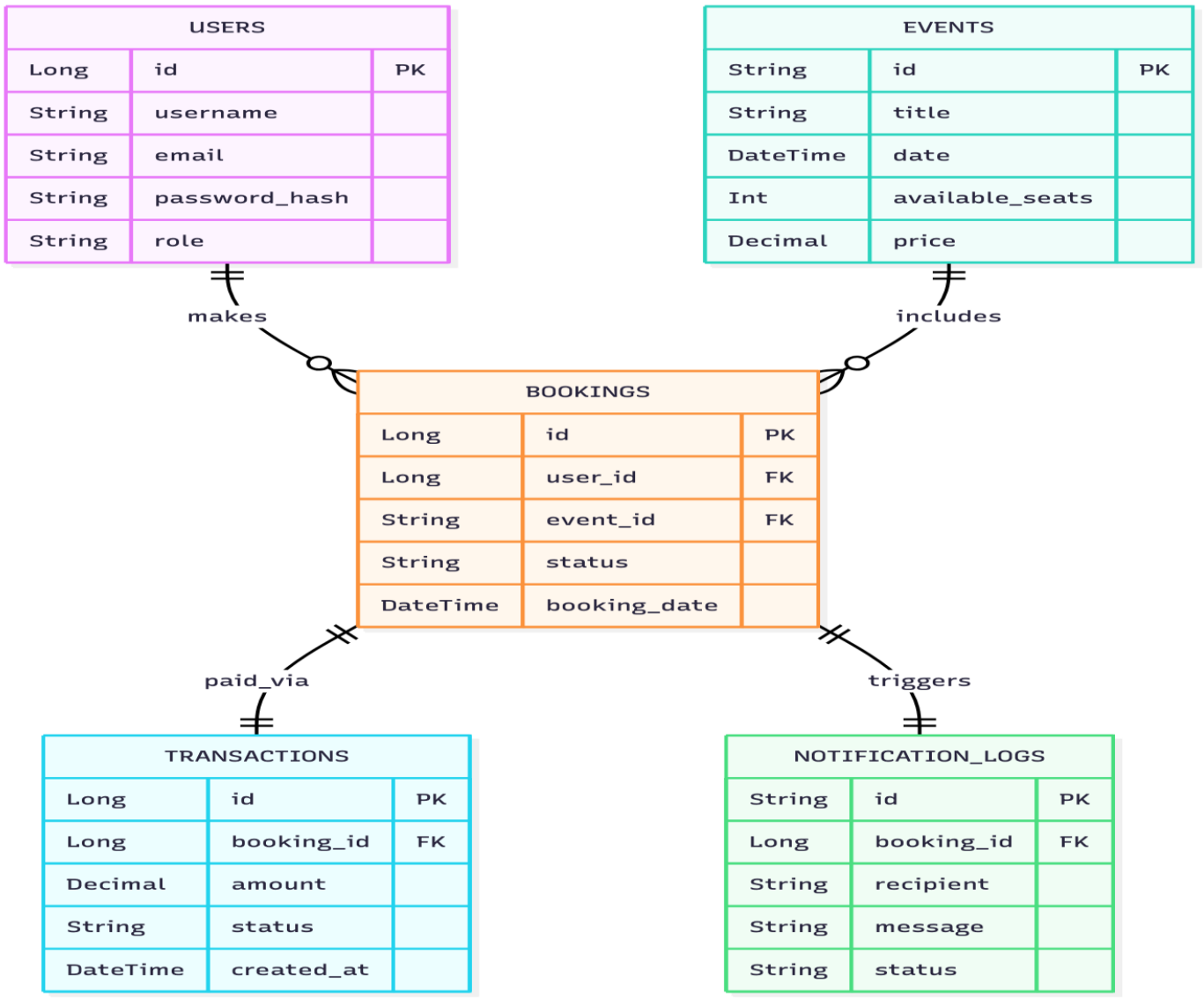
This sequence diagram illustrates the "Ticket Booking" flow. The process begins with a synchronous REST

request orchestrated by the Booking Service, which communicates with the Event and Payment services. Once the booking is confirmed, an asynchronous message is published to RabbitMQ, triggering the Notification Service to send an email without blocking the user response.

---

7. Data Model

Example ER Diagram (Mermaid)



Since this is a microservices architecture, there is no single shared database. Instead, each service manages its own data storage to ensure loose coupling:

- **Relational Data (SQL):** User, Booking, and Payment services use SQL databases (PostgreSQL/MySQL) to

ensure transactional integrity and ACID compliance for critical data.

- **Document Data (NoSQL):** Event Catalog and Notification services use MongoDB to handle flexible schemas and high read throughput requirements.
- **Relationships:** The lines in the diagram represent logical references (Foreign Keys) maintained at the application level, not physical database constraints.

8. Security & Operational Concerns

Authentication & Authorization:

- **JWT (JSON Web Token):** The User Service acts as the centralized identity provider. Upon successful login, it issues a signed JWT containing the user's ID and roles (e.g., ROLE_USER).
- **Gateway Validation:** The API Gateway intercepts all incoming requests and validates the JWT signature before forwarding the request to internal microservices (Booking, Event, etc.), ensuring only authenticated traffic enters the system.

Transport Security:

- **HTTPS:** All external communication between the client (Frontend) and the API Gateway is encrypted via TLS/HTTPS.
- **Inter-service Security:** Internal services are deployed within a private network on Render, preventing direct public access to databases or backend APIs.

Secrets Management:

- Sensitive data (e.g., database passwords, JWT signing keys, RabbitMQ URIs) are never hardcoded.
- They are managed securely using **Render Environment Variables** and injected into the application at runtime via the **Spring Cloud Config Server**.

Monitoring & Observability:

- **Metrics:** System health (CPU, Memory, Response Times) is monitored using the Render Dashboard.
- **Health Checks:** Each microservice implements Spring Boot Actuator (/actuator/health) endpoints, allowing the orchestration platform to automatically restart unhealthy instances.

Logging:

- **Centralized Logging:** All microservices write logs to standard output (stdout). These logs are aggregated by the Render platform, allowing developers to view and search logs from all services in a single console.

9. CI/CD Workflow

The project uses a continuous integration and deployment pipeline to ensure code quality and automated delivery.

9.1 Continuous Integration (CI) - GitHub Actions Every time code is pushed to the `main` branch, a GitHub Actions workflow automatically triggers to build the project and run unit tests. This ensures that no broken code reaches the deployment stage.

```yaml
name: Java CI with Maven

on: [push]

jobs:
  build:
    runs-on: ubuntu-latest

    steps:
    - uses: actions/checkout@v3

    - name: Set up JDK 23
      uses: actions/setup-java@v3
      with:
        java-version: '23'
        distribution: 'temurin'
        cache: maven

    - name: Build with Maven
      run: mvn clean package -DskipTests

    - name: Run Tests
      run: mvn test
```

9.2 Continuous Deployment (CD) - Render Cloud Deployment is managed via Render's Auto-Deploy feature, linked directly to the GitHub repository.

1. **Trigger:** When the GitHub Actions CI pipeline passes successfully, Render detects the change in the `main` branch.

2. **Build:** Render pulls the latest code and builds the Docker image using the `Dockerfile` present in the root of each microservice.

3. **Deploy:** The new container is deployed automatically, replacing the old instance with zero downtime.

1. Quick Checklist (for Students — Submit with SDD)

- ☐ Executive Summary included
- ☐ Functional & Non-functional requirements listed
- ☐ Architecture and diagrams attached
- ☐ Component descriptions for each microservice
- ☐ API specs for core endpoints
- ☐ Data model (ERD or collections) included
- ☐ Security and CI/CD considerations documented

- ☐ Test plan and sample results included
- ☐ Render service URL documented in README

---

2. Grading Rubric (10 Points Total)

Criterion	Description	Points
1. Clarity & Organization	Document is structured, readable, and follows template	**1**
2. Completeness	All major sections are filled with relevant content	**1**

Criterion	Description	Points
3. Architecture Design	Logical, modular design; diagrams well presented	**2**
4. Component Descriptions	Each microservice is clearly defined and justified	**1**
5. API Documentation	Clear endpoint specifications and data flow	**1**
6. Data Model Design	Well-structured ERD and relationships	**1**
7. Security & CI/CD Coverage	Key security and deployment details present	**1**
8. Testing Strategy	Realistic and traceable testing approach	**1**
9. Professional Presentation	Neat formatting, diagrams, and references	**1**

> 💡 *Instructor Tip:* Minor deductions (0.25–0.5 pts) can be applied for missing diagrams, unclear writing, or poor alignment between architecture and requirements.

Total: 10 points

Prepared for: COMP 301 — Fall 2025
Template version: 2025-10-25